

Go

Reading

For general help about Go, check out the Go tutorial getting started page.

Tour of Go

Please read the following parts of the Go tour (try the coding exercises they give as practice):

- **Read this:** Packages, variables, and functions
- **Read this:** Flow control statements: for, if, else, switch and defer
- **Read this:** More Types: structs, slices, and maps
- **Read this:** Methods and interfaces

Go Lectures

Lecture 1, 2: Go Basics

- Intro to Go

Careful! Go is finicky about the structure of files and folders: - The examples folder is a module. Note the file `go.mod` in that folder. - Individual programs within it have their own folders. - Within those folders, the program with the `main` function is called `main.go`. - Within the folder, you run the program with `go run main.go`. This compiles and runs the program.

Overview of Go Go is an application-oriented programming language developed by programmers at Google. One of the major use cases for Go is writing *servers*, and so much of the language is written to support this.

Much of what follows is based on the Go FAQ. This is a good starting place for learning about the design of Go.

Some of the major features of Go are:

- **Fast compilation.** Go is designed from the ground up for writing large, multi-file programs. In languages like C and C++, compiling and linking multi-file programs is surprisingly time-consuming due in large part to the use of `#define` to combine files. Many large C++ programs can't be efficiently built on a single computer, and so sophisticated distributed build systems are used to make build times more reasonable.
- **Lightweight typing.** Go is a *statically typed* language, like C++ or Java. Yet this is done in such a way that you can often avoid explicitly dealing with types, and so it feels closer in spirit to non- statically typed languages, like Python or JavaScript.

- **Novel use of interfaces.** Go is *object-oriented*, but it does not have classes or inheritance (at least in the sense of C++ or Java). Instead, interfaces and methods combine to provide most of the same benefits of traditional object-oriented programming.
- **Garbage collection.** As in Java or Python, there's no need to explicitly de-allocate memory. This simplifies code, and goes a long way towards stopping memory leaks, preventing dangling pointers, and enabling concurrent programming.
- **Closures.** Go supports lets you pass and return functions, and implements closures (i.e. functions that can refer to variables outside of themselves).
- **Concurrency support.** Go uses channels and so-called “go routines” (essentially lightweight processes) to handle most concurrency tasks. These are based on a model of concurrency known as communication sequential processes.
- **No exceptions.** The designers of Go believe that exception systems in languages like C++ and Java ultimately lead to convoluted code that treats too many situations as errors (e.g. they believe that a failure to open a file should not be treated as an exception). Instead, Go relies on explicit error codes returned from functions, along with the functions `defer`, `panic`, and `recover`.
- **Pretty good standard tools and library.** Out of the box, Go comes with useful tools for things like source code formatting (`go fmt`) and testing (`go test`). It also has an extensive standard library with many practical packages. For instance, it is relatively easy to create a simple web-server in Go using just its standard library.

Here are a variety of examples of Go programs. You should study them all to get a feel for the language.

- `hello_world.go`
- `hello_name.go`
- `age.go`
- `count_up.go`
- `count_down.go`
- `functions.go`
- `if_statements.go`
- `switch_statements.go`
- `primes.go`
- `defer.go`

Lecture 3, 4: Arrays, Slices, and Maps

Go has unusual implementing of lists. Go has **arrays**, which are like C arrays. Go arrays are a contiguous block of memory, and they can't be made bigger or

smaller. This is simple but inflexible.

Go also has **slices**. Slices are like “fat pointers” to arrays. A slice points to location in an underlying array, and also contains the length of the slice. In most Go programs, slices are the main way to implement dynamic lists (as in Python).

Go’s slice notation is similar to Python’s slice notation, but extending and modifying arrays is a bit different.

Since Go slices are like pointers, when you pass a slice to a function, the slice is copied but the underlying array is not copied. It is possible to have two, or more, slices that point to the same underlying array, and it is up to the programmer to keep track of this.

- slices.go
- bits.go
- sort.go
- wordcount.go
- functionsAndClosures.go
- compose.go

Lecture 5, 6: Types, Methods, and Interfaces

- point.go
- Please see the notes in interfaceDemo.md and the associated code in interface_Counter.go
- shapes.go

Lecture 7: Generics and Concurrency

Generics are a way to write code that can work with any type.

- generic_Index.go
- generic_stack.go

Concurrent programming is one of Go’s major strengths (perhaps *the* major strength). Concurrent programming lets you write programs that have multiple threads of control. This is a difficult topic to cover in even a couple of lectures, so we will only show a couple of basic examples.

- spinner.go
- gen.go